Ethics, Law & Responsible Data Collection

Week 2 · Foundations module · Textbook Chapter 2

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

August 24, 26 & 28, 2026

This week at a glance

This week covers the rules that govern collection: what is legal, what a site permits, and what is ethical.

Monday | Concepts & notebook intro

- The legal landscape, `robots.txt`, and a five-question ethical framework

Wednesday | Notebook lab

- `robots.txt`, User-Agent headers, rate limiting — and building `responsible_get()`

Friday | Textbook revisions

- GitHub, hands-on — and everyone files their first issue against Chapter 2

Companion notebook

`ch-02-ethics.ipynb`

Bring a laptop

Wednesday is hands-on — we send live requests and watch what the server sees.

1. Monday · Concepts

Agenda

00:00 – 00:04	The ethics of retrieval
00:04 – 00:20	The law: the CFAA, four rulings, and what is still moving
00:20 – 00:26	Terms of Service & privacy policies
00:26 – 00:34	**Activity**: audit a platform you use
00:34 – 00:44	**Discussion**: four articles on AI scraping
00:44 – 00:50	The framework, and Wednesday

Before we start

A browser, and the four articles from the reading. You will read primary sources for about half of today.

The ethics of retrieval

"Web scraping" is a popular phrase — and a faintly pejorative one. Data is **valuable**. Other people spent time, money, and effort to create and host what you are about to retrieve.

Ask two questions, in order. **Can I scrape this?** Then: **should I, and how do I do it responsibly?**

This chapter builds a **decision framework** to revisit before every collection project in this course and beyond. Three layers of guidance, none sufficient alone: **law**, **technical norms** (`robots.txt`), and **ethics**.

Key idea

"It's technically possible" is not the same as "it's legal," and "it's legal" is not the same as "it's ethical."

Textbook Ch. 2, "The Ethics of Retrieval."

The Computer Fraud and Abuse Act (1986)

The **Computer Fraud and Abuse Act** was enacted in 1986, expanding a narrower 1984 statute, during a national panic about "computer crime." The web did not exist.

Its key provision prohibits **"unauthorized access"** to a computer system. What the statute never does is say what makes access to a *public web page* unauthorized.

The CFAA is a **criminal** statute. That makes the question "was this access authorized?" the difference between a research method and a federal charge.

Courts have argued over that phrase for forty years.

Relevance to your work

Every collection decision this semester falls under a criminal statute written before the web existed. Courts still disagree about how it applies.

Textbook Ch. 2, "The Computer Fraud and Abuse Act."

Aaron Swartz

Aaron Swartz was an influential hacker from the 2000s: RSS, Creative Commons, web.py, Markdown, Reddit

In 2010 and 2011, Swartz used MIT's open network to bulk download academic articles from JSTOR *maybe* with the intent to release scholarship from behind the paywall

Federal prosecutors charged him in 2011 under the CFAA's "unauthorized access" rule carrying several decades of jail time

He died by suicide in 2013, at 26.

"The Internet's Own Boy: The Story of Aaron Swartz" (2014)

Textbook Ch. 2, "Social History and Public Interest." On the CFAA's use against researchers: Fiesler et al. (2020). Photo: Fred Benenson, CC BY 2.0.

Cambridge Analytica

Cambridge Analytica was a political consulting firm that claimed it could target voters by personality. Clients included the 2016 Trump campaign and Leave.EU

In 2014, Cambridge psychologist Aleksandr Kogan ran a personality quiz app on Facebook. About 270,000 people took it. The Graph API also gave him their friends' data: roughly 87 million profiles. He passed all of it to Cambridge Analytica, breaking Facebook's platform rules, *not* the CFAA

Christopher Wylie disclosed it in March 2018. The firm closed that May. The FTC fined Facebook $5 billion in 2019

Facebook and other platforms then cut off API access. Researchers who broke no rules lost it too

"The Great Hack" (2019)

Christopher Wylie, who disclosed the operation.

Textbook Ch. 2, "Social History and Public Interest." The closure that followed is Ch. 3. Photo: Chatham House, CC BY 2.0.

Van Buren v. United States (2021)

The facts. A police sergeant ran a license-plate search in a database he was *authorized* to use — but did it for a bribe, not police work. He was charged under the CFAA's "exceeds authorized access" clause.

The holding. The Supreme Court adopted a **"gates-up-or-down"** reading: the CFAA targets entering areas you are not entitled to enter. It does *not* reach misusing information you were allowed to see.

Why it matters to you. Before *Van Buren*, a prosecutor could plausibly argue that breaking a website's stated terms was a federal crime. After it, that theory is very hard to sustain.

The test

Did you get past an authentication barrier, or did you break a rule about data you could already reach?

Only the first is a CFAA question.

Textbook Ch. 2, "Legal Frameworks." A gate is authentication — a login, a paywall, an API key.

hiQ Labs v. LinkedIn (2019–2022)

The case has three parts, and the third is usually left out:

1. hiQ, an analytics company, scraped **public** LinkedIn profiles. LinkedIn sent a cease-and-desist and blocked it.
2. The Ninth Circuit held — in 2019, and **again in 2022** after the Supreme Court sent it back in light of *Van Buren* — that scraping publicly accessible data likely does **not** violate the CFAA. No authentication gate was bypassed.
3. Later in 2022, hiQ **lost** on LinkedIn's **breach-of-contract** claim: it kept scraping after agreeing to the ToS and receiving the C&D. The case settled with hiQ agreeing to stop.

A common misreading

A common summary is "*hiQ* means scraping public data is legal."

That describes only the second part. hiQ still lost and stopped scraping.

The CFAA is not the only law in the room.

Textbook Ch. 2, "Legal Frameworks." Criminal exposure and contractual exposure are different questions with different answers.

Sandvig v. Barr (2020)

The facts. Researchers wanted to run **audit studies** of algorithmic discrimination. They planned to make test profiles on housing and employment sites. The profiles would show if the algorithms treated people differently by race. This method violates the Terms of Service of those sites. The researchers feared prosecution under the CFAA, so they sued *first*.

The holding. A federal district court held in 2020 that merely violating a website's Terms of Service is **not** a CFAA crime.

Why it matters. Audit studies are the main source of evidence about algorithmic discrimination. This ruling is from one district court, so it is limited, but it protects the method.

The underlying tension

Platforms write the rules that govern how researchers may study them.

Terms of Service that forbid test accounts also prevent an audit for discrimination.

Textbook Ch. 2, "Legal Frameworks." Audit methodology and ToS restrictions: Fiesler et al. (2020).

Summary of the rulings

Case	Holding	What it means for you
Van Buren (2021)	CFAA reaches gates, not use restrictions	Breaking a rule ≠ a federal crime
hiQ (2019, 2022)	Public scraping likely not CFAA — but breach of contract	Winning on the CFAA is not winning
Sandvig (2020)	ToS violation alone is not a CFAA crime	Audit research has some cover
Bright Data (2024)	Logged-out public scraping survived contract claims	Never agreeing means no contract

Summary: the CFAA covers **authentication barriers**, not **use restrictions**. What left criminal law did not disappear. It moved to **contract law**, which changes the court and the penalty, not whether you are exposed.

Textbook Ch. 2, "Legal Frameworks" & "The Current Landscape."

2024: Bright Data and the limits of contract

Two rulings against the same data broker mark where the line is now:

- **Meta v. Bright Data** (early 2024) — Meta's breach-of-contract claims **failed**. The scraping at issue collected only public data *without logging in*, so the account terms Meta relied on never attached.
- **X Corp. v. Bright Data** (later 2024) — **dismissed**, with the court warning against letting platforms use contract law to claim exclusive ownership of public web data.

These are trial-court rulings, not nationwide rules. But the direction is consistent with *hiQ*: risk concentrates at **authentication barriers and contractual relationships**, not at public visibility.

The key distinction

Logged out: you agreed to nothing, so there is no contract to breach.

Logged in: you agreed, and the Terms of Service are enforceable against you.

The data is the same. The legal exposure is not.

Textbook Ch. 2, "The Current Landscape."

The AI front: *NYT v. OpenAI*

The New York Times v. OpenAI and Microsoft (filed 2023) asks a question that no earlier case asked. Many similar suits ask it too. Is it infringement to collect copyrighted content and **train a model** on it? Or is it fair use?

That question is unresolved. But the *litigation* has already changed the web, without waiting for a ruling:

- publishers hardened their sites against crawlers
- AI-specific directives appeared in `robots.txt`
- exclusive licensing deals carved up access

Effect on your work

You are not a party to these suits, but the result affects you.

A site that blocks OpenAI also blocks your class project. Ch. 3 covers this as **enclosure**.

Textbook Ch. 2, "The Current Landscape." Data revolts against AI scraping: (Frenkel et al. 2023).

State privacy law

The U.S. has no comprehensive federal privacy law, so states filled the gap. California wrote the **CCPA/CPRA** first. By 2026 more than a dozen states have comprehensive statutes. They include the **Colorado Privacy Act**, which covers data about Colorado residents.

They import GDPR-like concepts: **personal data rights**, **purpose limitation**, and obligations on whoever processes the data.

The **GDPR** itself reaches EU residents' personal data regardless of where *you* are sitting.

If your scraped dataset contains personal information about residents of these places, these laws can apply to **you** — student status is not an exemption.

Practical rule

Treat personal data as a **liability**.

Each extra field is more risk that you have chosen to hold. Collect the minimum that answers your question.

Textbook Ch. 2, "The Current Landscape" & "International Considerations."

Terms of Service as quasi-law

As CFAA exposure narrowed, contract law expanded. The Terms of Service is now the document that constrains your work.

Every major platform's ToS restricts automated collection. When you created an account, you agreed. Consequences run from account suspension to IP blocking to litigation.

The agreement is also **asymmetric**. You accepted it once by clicking. The platform amends it by posting a new version. Tan (2024) documents platforms rewriting terms and privacy policies to permit **AI training** on data that users had already shared.

Course warning

Code in several chapters retrieves data in ways that may violate a platform's ToS. It is here to teach technique.

You are responsible for the terms of anything you access. When in doubt: use the API, or don't collect it.

Textbook Ch. 2, "Terms of Service as Quasi-Law." ToS rewritten for AI training: Tan (2024).

Reading a ToS or privacy policy

These documents are long, and few people read them from start to finish. Search them instead.

Keywords worth a Ctrl-F:

`access` | `script` | `scrape` | `crawl` | `automated` | `bot` | `API` | `research` | `data` | `abuse` | `rate` | `machine learning`

Four questions to come out with:

1. Is **automated access** addressed at all?
2. Is there a **research or academic** exception?
3. What does it say about **personal data** and **AI training**?
4. When was it **last updated** — and what changed?

Note on question 4

Tan (2024) found changes as small as a few words — inserting "artificial intelligence" or "machine learning" into a policy you agreed to years ago.

In February 2024 the **FTC** warned that rewriting a privacy policy to retroactively cover data already collected may be "**unfair or deceptive**."

Method adapted from the prior offering of this course. ToS drift: Tan (2024).

Activity: audit a platform you use

Eight minutes. Select a platform that you use. Open its Terms of Service or its privacy policy.

1. Search the document for the keywords on the last slide.
2. Answer the four questions.
3. Find one clause that surprised you.

Then report two things. First, does this platform permit the data collection that you plan for this class? Second, how sure are you?

Public institutions also have terms

The Colorado General Assembly publishes every bill, sponsor, and vote — and still has a privacy policy governing its site.

"Public data" never means "no terms." It is the first place students assume otherwise.

Report back in pairs. We will collect the surprising clauses on the board.

Discussion: four articles on AI scraping

Ten minutes. Four minutes in groups, then six minutes for reports. Each group takes one article:

1. **The terms move —** Tan (2024), on ToS rewritten to permit AI training
2. **The web pushes back —** Koebler (2024a), on measurable mass blocking
3. **The defense misfires —** Koebler (2024b), on sites blocking the wrong bots
4. **The rules get ignored —** Mehrotra et al. (2024), on scraping around a block

Questions for each group

1. What **changed**, concretely?
2. Who **gains** and who is **harmed**?
3. What does it mean for **your** project this semester?

All four are on Canvas. Question 3 is the one I will push you on.

Common findings

The four articles report four related changes:

1. **Terms change.** Platforms amend them to permit AI training on data that users already shared. In February 2024 the FTC warned that doing this retroactively may be “unfair or deceptive” (Tan 2024).
2. **Sites add restrictions.** Of 14,000 sites studied, full AI restrictions went from about 1% to 5–7% in one year. **28%** of the most actively maintained sources added restrictions (Koebler 2024a).
3. **The blocks often fail.** Reuters and Condé Nast blocked two *retired* Anthropic crawlers but not the active one. iFixit received about 1 million requests in one day. Read the Docs spent $5,000 on bandwidth (Koebler 2024b).
4. **Some operators ignore the rules.** Perplexity reached blocked pages from an unpublished IP address. It then fabricated a claim that a named police officer had committed a crime (Mehrotra et al. 2024).

Effect on student projects

A blanket block cannot distinguish an AI crawler from a search engine, an archive, or a student project. Restrictions aimed at AI companies also apply to your coursework.

An ethical decision framework

Before any project, work through five questions:

1. **Legality** — CFAA, GDPR, local law, the site's ToS?
2. **Consent** — did people consent? Is it truly public, or shared with a reasonable expectation of limited visibility (Zimmer 2010)?
3. **Proportionality** — only the data you need? Could a smaller or less sensitive set answer the question?
4. **Privacy** — any PII? Could individuals be re-identified? What if it leaks?
5. **Server impact** — a meaningful load? Rate-limited? Scraping off-peak?

No framework substitutes for judgment — but working these questions surfaces the risks.

IRBs & web data

Scraping becomes *human-subjects research* when you collect data about **identifiable people** to produce **generalizable knowledge**. Public officials' votes: usually not. Social-media profiles for health or politics: almost certainly. When in doubt, ask your IRB.

Textbook Ch. 2, "An Ethical Decision Framework" & "IRBs and Web Data." The Belmont Report maps consent → respect, proportionality/privacy → beneficence, risk → justice.

Wednesday: `robots.txt`

Today covered the **law** and the **contract**. Wednesday covers the third layer, the **technical norm**, in code.

You will get the `robots.txt` file of many real sites and compare them. Then you will build the `responsible_get()` function. You will use that function for the rest of the semester.

Come with a prediction. Pick two sites you would expect to differ — a nonprofit and a newspaper, say — and guess which restricts more, and why. We will check.

Bring a laptop

We send live requests and watch what the server sees. Your `webdata` environment should already work — if it does not, fix it **before** Wednesday.

Read Textbook Ch. 2's "Technical Norms" section before Wednesday, plus the four articles for Monday's discussion.

2. Wednesday · Notebook Lab

`robots.txt`: read it before you scrape

A `robots.txt` at a site's root declares what automated agents may fetch. Read it raw, or parse it and ask permission per URL:

```python
import requests
from urllib.robotparser import RobotFileParser

print(requests.get("https://en.wikipedia.org/robots.txt").text[:400])
# User-agent: * / Disallow: /w/ / Allow: ... / Crawl-delay: ...

rp = RobotFileParser()
rp.set_url("https://en.wikipedia.org/robots.txt")
rp.read()

rp.can_fetch("*", "https://en.wikipedia.org/wiki/Python_(programming_language)") # True
rp.can_fetch("*", "https://en.wikipedia.org/w/api.php") # False
rp.crawl_delay("*") # seconds between requests, or None -> pick a polite default
```

The nuance: `/wiki/` articles are open, but `/w/` (dynamic pages + API) is disallowed *for crawlers*. Deliberate API clients follow different norms (Ch. 10) — `robots.txt` is one input, not the whole rulebook.

Textbook Ch. 2, "Technical Norms: `robots.txt`."

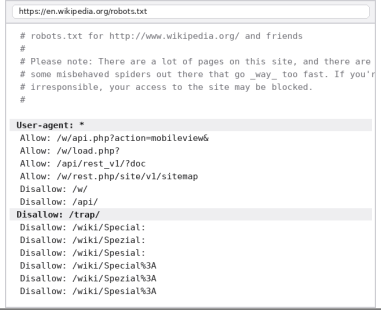
What `robots.txt` reveals across platforms

```
sites = {"Wikipedia": "https://en.wikipedia.org/robots.txt",
         "Reddit": "https://www.reddit.com/robots.txt",
         "NY Times": "https://www.nytimes.com/robots.txt"}

for name, url in sites.items():
    lines = requests.get(url).text.strip().split("\n")
    disallow = sum(l.startswith("Disallow") for l in lines)
    allow = sum(l.startswith("Allow") for l in lines)
    print(f"{name}: {len(lines)} lines, {disallow} Disallow, {allow} Allow")
```

Wikipedia — permissive where it matters (openness is the mission; data flows through its API + dumps). **Reddit** — tolerates access within limits. **NY Times** — restrictive; the content is a commercial product. Reading it tells you *what kind of organization* you're dealing with.

Two cautions. First, it is a **norm, not a law**: a violation is not illegal, but it is impolite and the site can block your IP address. Second, a **404** means *no rules*. It does not mean "crawl aggressively."



Textbook Ch. 2, "Comparing `robots.txt` Across Platforms."

Activity: survey a sector

Three sites tell you little. A whole sector shows you how a business model affects access. Select one row. Get the `robots.txt` file of each site in that row. Compare them:

- **Search** — `google.com`, `bing.com`, `duckduckgo.com`
- **Social** — `facebook.com`, `instagram.com`, `linkedin.com`, `x.com`, `reddit.com`
- **Commerce** — `amazon.com`, `ebay.com`, `ticketmaster.com`
- **Streaming** — `netflix.com`, `hulu.com`, `spotify.com`
- **News** — `nytimes.com`, `washingtonpost.com`, `wsj.com`, `theguardian.com`
- **Public** — `leg.colorado.gov`, `census.gov`, `sec.gov`

Report back

- Most and least restrictive in your row?
- What does the pattern say about **how each makes money**?
- Does the **public** row look different — and should it?

Sector list adapted from the prior offering of this course. Count `Disallow` vs `Allow` lines with the loop from the last slide.

Which AI crawlers does this site block?

Since 2023, `robots.txt` has grown a second job: naming **AI training crawlers**. Search each file you fetched for:

`GPTBot` (OpenAI) | `ClaudeBot` (Anthropic) | `CCBot` (Common Crawl) | `Google-Extended` | `Applebot-Extended` | `Meta-ExternalAgent` | `PerplexityBot`

Then look for the error that Koebler (2024b) found: a site blocks a **retired** crawler name, but not the operator's **active** one. Copied blocklists go out of date.

`robots.txt` also has no enforcement mechanism. Mehrotra et al. (2024) documents a company that reached blocked pages from an unpublished IP address.

Why respect it

Because your goal is defensible research, not avoiding detection.

The norm does not stop bad actors, but ignoring it will damage your reputation.

Crawler-name rot: Koebler (2024b). Ignoring the protocol outright: Mehrotra et al. (2024).

Identify yourself, then slow down

Two habits make you a good citizen of the web — a truthful `User-Agent` and rate limiting:

```python
import time, requests

headers = {
    "User-Agent": "WebDataScience/1.0 (INFO 4617; brian.keegan@colorado.edu)"
}

for url in urls:
    response = requests.get(url, headers=headers)
    # ... process the response ...
    time.sleep(1) # a good default: about one request per second
```

The default `requests` User-Agent marks you as an anonymous bot. A good one **identifies you and gives contact info**, so operators can reach you instead of simply blocking you — and some APIs *require* it. Hundreds of requests per second is an accidental **denial-of-service**: respect any `Crawl-delay` and stay well inside documented API limits.

Your header is visible to every server you touch.

Textbook Ch. 2, "Responsible Request Headers" & "Rate Limiting." HTTP mechanics: *Missing Manual* Ch. 24.

`responsible_get()`: build it once

Custom User-Agent, rate limiting, and error handling appear in *every* scraping project. Wrap them into one reusable function instead of re-implementing them ad hoc:

```python
import requests, time

def responsible_get(url, headers=None, delay=1.0):
    """GET with responsible defaults: identify, rate-limit, fail loudly."""
    default_headers = {
        "User-Agent": "WebDataScience/1.0 (INFO 4617; you@colorado.edu)"}
    if headers:
        default_headers.update(headers)
    try:
        response = requests.get(url, headers=default_headers, timeout=30)
        response.raise_for_status() # treat 4xx/5xx as failures, not data
        time.sleep(delay) # rate-limit by default
        return response
    except requests.exceptions.RequestException as e:
        print(f"Request failed ({type(e).__name__}): {url}")
        return None
```

Textbook Ch. 2, “Building a Responsible Scraper Function.”

Design choices, and later reuse

Every default encodes a principle

- **Custom User-Agent** — identify & give contact
- `delay` **defaults ON** — aggressive scraping must be a *conscious* override, not an oversight
- `raise_for_status()` — a 403/404 fails loudly instead of feeding your parser garbage
- broad `except` — `ConnectionError`, `Timeout`, `SSLError` log rather than crash the run
- `timeout=30` — never hang forever on a dead server
- pass a site's `Crawl-delay` as `delay` to honor its stated preference

Reused in later chapters

`responsible_get()` returns in **Ch. 6** (scraping HTML tables), **Ch. 7** (Wayback snapshots), and throughout the **API chapters**.

Building responsible defaults into your *tools* is more reliable than remembering to add them each time.

Textbook Ch. 2. Reused in later chapters on static pages, archives, and APIs.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Compare the `robots.txt` files of five sites in one category. Report the patterns that you find.
2. Read one Terms of Service. Find three clauses that limit automated collection. Report if research is an exception.
3. Apply the five-question framework to this plan: collect dating-app profiles to study self-presentation.
4. Measure five requests with `time.time()`. Use a 1-second delay, then a 2-second delay. Compare the times with your intent.
5. Select a site that you use each day. Compare its `robots.txt`, its Terms of Service, and the framework. Write 300 words. Show where they disagree.
6. **5617**: read Fiesler et al. (2020), the EFF page about the CFAA, and *Van Buren*. Then write a two-page memo about collection from platforms

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

“Can I scrape this?” is the wrong question.

Ask instead: *should* I — and if so, how do I do it responsibly?

Legality · Consent · Proportionality · Privacy · Server impact.

3. Friday · Textbook Revisions

Friday: your first contribution

Week 1 was a single short session and we ran out of room before anyone filed anything. So today we do it properly, and slowly.

By the end of class you will have:

1. a working GitHub setup, verified with me in the room
2. **one real issue filed** against Chapter 2

Pull requests start **next week**. Today is the cheaper, faster half of contributing — and it is the half that finds the problems.

00:00 – 00:06	The four objects
00:06 – 00:18	Setup, together
00:18 – 00:26	What to look for
00:26 – 00:32	Anatomy of an issue
00:32 – 00:47	File it
00:47 – 00:50	Next week

Contributions & reviews count toward the Textbook Revisions grade (15%).

Four objects you'll use all semester

- **Repository —** the project: every file and its full history.
- **Issue —** a written report: "here is a problem." Costs nothing but a browser.
- **Pull request —** a proposed change: "here is the fix, side by side with the original."
- **Review —** a response to a PR: comment line-by-line, then approve or request changes.

Issue vs. pull request

An **issue** says *something is wrong*. A **pull request** says *here is the correction* — and shows the exact diff.

Issues are cheap and fast; PRs are the real contribution. **Today: an issue.** From next week on: pull requests.

This is how essentially all open-source software gets built — including every library in this course.

Setup: two options

To file an issue you need only a browser. That is today's task, and it works for everyone.

We will also set up your **tools** now, while I am here to help. Next week you need them for pull requests. Select one tool:

GitHub Desktop — download it, sign in, then **Clone** the textbook repository. Each later step is a button: branch, commit, push, Create Pull Request.

The `gh` CLI — install it, run `gh auth login`, then use these commands:

- `gh repo clone cuinfoscience/Web-Data-Science-Book`
- `gh issue create` (yes, you can file today's issue this way)

Checkpoint

You are done when Desktop shows the textbook repository on your computer, or when `gh repo clone` completes with no error.

Tell me if a step fails. Today is the one class period when I can repair it for you.

Full instructions, both paths: `handouts/week-01/setup.md`. No raw `git` commands required in this course.

Revision targets for Chapter 2

High-value targets — pick one and scope it to a single PR:

- **A confirmed gap.** Wikipedia's live `robots.txt` now has a `Disallow: /trap/` rule the chapter doesn't mention — a honeypot for bad bots. Explain what it's for and add it to the comparison.
- **Close a gap in "Common Issues to Debug."** Not covered yet: a blocked page can return **HTTP 200 with a Cloudflare challenge page**, not a 403 — `raise_for_status()` won't catch it. Add this as a worked case.
- **Add a figure — there are currently zero in this chapter.** A clean five-question framework diagram, an annotated `robots.txt` anatomy, or a CFAA case timeline.
- **Add a missing citation.** The Meta v. Bright Data (2024) and X Corp. rulings are described in prose with no source link — find and add one.
- **Strengthen an exercise.** Add a rubric or expected output to the framework exercise (#3), or a starter cell so it self-checks.

Targets confirmed against the live chapter and current `robots.txt` — not hypothetical.

Anatomy of a good issue

Every issue you file this semester uses one skeleton:

```
Title: <type>: <specific thing> in Ch. 2
Location: Section, and the file
          (ch-02-ethics.qmd)
Problem: What you did, expected, got.
         Paste the code and output.
Why: Who this affects. One sentence.
Proposal: The concrete change you'd make.
```

If you cannot fill in **Location** and **Problem** with specifics, you do not have an issue yet — you have a hunch. Go back and reproduce it.

Scored out of 10

- **Located** (2) — section and file named
- **Evidenced** (3) — I can hit the same wall
- **Actionable** (3) — one concrete change
- **Reviews** (2) — from next week on

Grading does not depend on merging

A well-evidenced issue I decline still earns full credit. A one-character typo fix does not.

Full rubric and worked examples: `handouts/common/revision-framework.md`

File it: your first issue

Fifteen minutes. Each student files one issue.

1. Read the chapter: `https://cuinfoscience.github.io/Web-Data-Science-Book/ch-02-ethics.html`
2. Find one problem. Use the revision menu, or use a different problem that stopped you.
3. Do the step again and record the result. Copy the passage, the command, or the output.
4. Select the **Issues** tab. Select **New issue**. Complete the skeleton. Select **Submit**.
5. Copy the URL of your issue into the Canvas assignment.

Read the open issues first. If another student filed your issue, add a comment to it. Write what that issue does not include. This also gets credit.

One browser tab. No install required.

Due tonight at midnight if you do not finish in class — but the point is that you finish in class.

Next week: from issues to pull requests

An issue says *something is wrong*. Starting next week you say *here is the fix* — and back it with a diff.

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **The Post-API Age** — why the open web is closing, and how researchers adapt.

Key takeaways

1. **"Can I?" is not "should I?"** — the real question is how to collect data *responsibly*.
2. The law is unsettled: the CFAA targets **bypassing gates** (Van Buren), but **contracts and ToS** still bite (hiQ); state privacy laws and GDPR reach personal data.
3. `robots.txt` is a **norm, not a law** — read it, respect `Disallow` and `Crawl-delay`; a 404 means no rules, not open season.
4. **Identify yourself** (custom `User-Agent`) and **rate-limit** (`time.sleep`) — then wrap both into `responsible_get()` and reuse it.
5. Run the **five-question framework** before every project; when in doubt, use an API, collect less, and err toward caution.

Reading: Textbook Ch. 2. Related: Fiesler et al. (2020); Freelon (2018).